



AT ALL NETWORKS

Algorithms & Complexity

Program



AT ALL NETWORKS

Python syntax

Functions

Modularity

Data structures (list, dict, set, tuple)

Git & Github

Algorithms & Complexity

Problem solving





AT ALL NETWORKS

Complexity

Big O

Computational Complexity



AT ALL NETWORKS

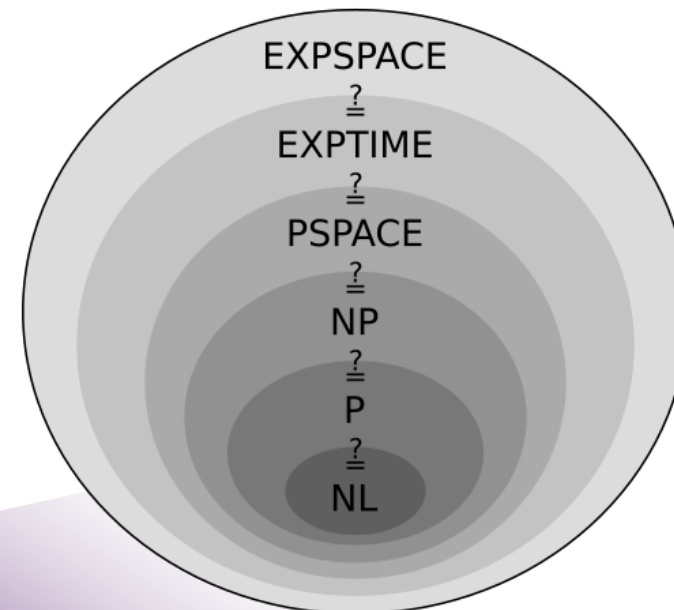
Computational complexity is the study of how the resources (time and space) required to run an algorithm grow as a function of the input time.

- **Time** → number of operations
- **space** → memory used

It describe the **rate of growth** using **Big O notation**, giving a **machine-independent** way to **compare algorithms**.

An algorithm is considered efficient if its resource usage grows relative to n .

Problems are grouped into **classes** based on how complicated they are.



Complexity classes



AT ALL NETWORKS

Class P

P is the category which have algorithm that can be computed in **polynomial** time.

→ class P refers to all problem that a computer can solve in a **reasonable amount of time**.

For n^2

- **n** is the number of **inputs** to the problem.
- The number of steps the algorithm needs to take is **proportional**.

Class NP

NP are problems that can be verified in **polynomial** time but may take an **exponential** number of steps to solve.

→ class NP refers to problems that take a **very long time** for a computer to solve.

2^n

- **n** is the number of **inputs** to the problem.
- The number of steps the algorithm needs to take can be **exponential**.



NP-complete problems are the hardest problems in NP, and they take the longest to solve.

Big O or Landau's notation



AT ALL NETWORKS

Big O notation is a way to describe how will an algorithm **perform** as its input size grows.

Notation	Name	Example	Scales to $n = 1,000$?
$O(1)$	Constant	Dict lookup, array index	✓ trivially
$O(\log n)$	Logarithmic	Binary search	✓ ~10 ops
$O(n)$	Linear	Single loop	✓ 1,000 ops
$O(n \log n)$	Linearithmic	Merge sort, sorted()	✓ ~10,000 ops
$O(n^2)$	Quadratic	Nested loops, bubble sort	⚠ 1,000,000 ops
$O(n^3)$	Cubic	Triple nested loops	✗ 1,000,000,000 ops
$O(2^n)$	Exponential	Naïve recursion, brute-force subsets	✗ astronomical
$O(n!)$	Factorial	Brute-force permutations	✗ incomputable

Big O



AT ALL NETWORKS

Basic summary of some operations and their complexity

Operation	Time	Space
Access array by index	$O(1)$	—
Loop through n elements	$O(n)$	—
Nested loop (both over n)	$O(n^2)$	—
Binary search / halving	$O(\log n)$	—
Sorting (general)	$O(n \log n)$	—
Dict / set lookup	$O(1)$ average	—
Creating a new list/dict of size n	—	$O(n)$
A fixed number of variables	—	$O(1)$
Linear recursion (calls itself n times)	$O(n)$	$O(n)$ stack
Divide & conquer (halves each time)	$O(\log n)$	$O(\log n)$ stack

Analysing a function in 4 steps



AT ALL NETWORKS

Count the loops

- One loop = $O(n)$
- Nested = $O(n^2)$

Look for halvings

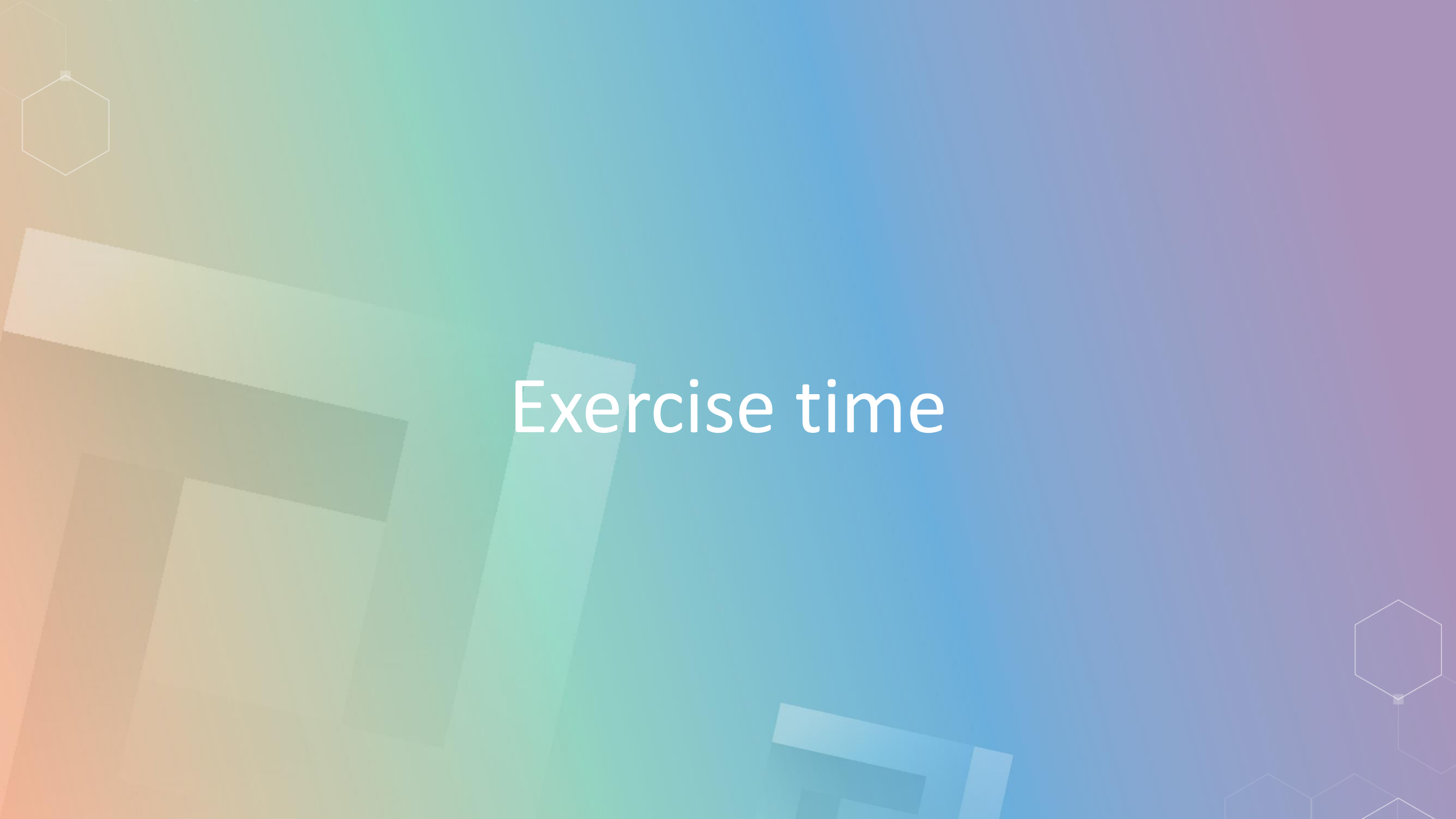
- $\text{mid} = (\text{lo} + \text{hi}) // 2$ inside a loop = $O(\log n)$

Look for recursive calls

- how many calls per level x how many levels = $O(n)$

Identify auxiliary structures

- a dict copy inside a loop = $O(n)$ extra space per call



Exercise time



AT ALL NETWORKS

Classical algoritms

Data structure vs algorithm



AT ALL NETWORKS

They are deeply linked — **the right data structure often makes an algorithm drastically faster.**

Two-sum is $O(n^2)$ with a list, $O(n)$ with a hash map. Same problem, different structure, completely different result.

Data structure



How you organize
data in memory

Algorithm



Procedure for
solving a problem

Fundamentals Data Structures



AT ALL NETWORKS

Structure	Internal Mechanism	Access	Insert	Delete	Key Use Cases
Array / List	Contiguous memory, dynamic resize ($\times 2$)	$O(1)$ by index	$O(1)$ amortized (end) / $O(n)$ middle	$O(n)$ middle	Foundation for everything, indexing
Hash Table dict / set	Hash function $\rightarrow$ array index, open addressing	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	Fast lookups, deduplication, replacing in on lists
Linked List deque	Nodes with pointers, non-contiguous memory	$O(n)$	$O(1)$ at head	$O(1)$ at head	BFS queue, undo systems, popleft() in $O(1)$
Stack list	LIFO — append / pop at top	$O(1)$ top	$O(1)$	$O(1)$	Undo, back button, parsing nested structures
Queue deque	FIFO — append at back, pop at front	$O(1)$ front	$O(1)$	$O(1)$	BFS, task scheduling, order-sensitive systems
Tree	Hierarchical nodes, BST adds ordering	$O(\log n)$ balanced	$O(\log n)$	$O(\log n)$	Filesystems, DOM, databases (B-trees), JSON
Heap heapq	Tree where parent $\leq$ children (min-heap)	$O(1)$ min/max	$O(\log n)$	$O(\log n)$	K largest/smallest, Dijkstra, priority queues
Graph	Adjacency list (dict) or matrix (2D array)	$O(1)$ – $O(n)$	$O(1)$ list	$O(E)$	Social networks, maps, dependency resolution

Classical algorithm



AT ALL NETWORKS

Search algorithms

- linear search
- binary search

Sort algorithms

- bubble sort
- selection sort
- insertion sort
- Merge sort
- Quick sort
- Heap sort
- Counting sort

Graphs/Tree traversal algorithms

- DFS (Depth-First Search)
- BFS (Breadth-First Search)

Graphs algorithms

- Dijkstra
- Bellman-Ford
- Floyd-Warshall
- A*
- Kruskal (also Greedy)
- Prim (also Greedy)
- Topological Sort

Classical algorithm



AT ALL NETWORKS

Dynamic programming

- Optimized Fibonacci
- Knapsack
- LCS (Longest Common Subsequence)
- LIS (Longest Increasing Subsequence)
- Coin Change
- Kadane

Backtracking

- N-Queens
- Sudoku Solver
- Permutations / Combinations

Greedy algorithms

- Activity selection problem
- Huffman coding
- Fractional knapsack

Number theory

- Sieve of Eratosthenes
- GCD (Greatest Common Divisor – Euclid's algorithm)
- Fast Exponentiation

String matching algorithms

- Rabin-Karp
- KMP (Knuth-Morris-Pratt)
- Z-algorithm

Must know



AT ALL NETWORKS

Algorithm	Category	Notes
Binary Search	Search	Asked constantly – know it by heart
Merge Sort	Sorting	Divide & conquer, stable $O(n \log n)$
Quick Sort	Sorting	Avg $O(n \log n)$, pivot/partition logic often asked
DFS	Graph / Tree traversal	#1 most used – paths, cycles, components
BFS	Graph / Tree traversal	Shortest path (unweighted), level-order
Dijkstra	Graph	Shortest path – very common in interviews
Coin Change	Dynamic programming	Classic DP intro question
Knapsack (0/1)	Dynamic programming	Core DP pattern – must understand
LCS / LIS	Dynamic programming	Very common in string/sequence problems
Permutations / Combinations	Backtracking	Extremely common in coding interviews
Huffman Coding	Greedy	Compression – often in CS fundamentals

Hight priority



AT ALL NETWORKS

Algorithm	Category	Notes
Linear Search	Search	Trivial but shows baseline understanding
Heap Sort	Sorting	Closely tied to heap data structure
Bubble / Insertion / Selection Sort	Sorting	Know complexities; rarely implemented in interviews
Topological Sort	Graph	Dependency problems, DAG questions
Kruskal / Prim	Graph	MST – mostly FAANG-level interviews
Kadane's	Dynamic programming	Max subarray – appears often
N-Queens / Sudoku	Backtracking	Classic backtracking – shows technique
Activity Selection	Greedy	Interval scheduling, very common pattern
GCD (Euclid)	Number theory	Appears in math-heavy interviews
KMP	String matching	Pattern matching – asked at top-tier companies

Good to know



AT ALL NETWORKS

Algorithm	Category	Notes
Counting Sort	Sorting	Good for bounded integers, rarely asked
Bellman-Ford / Floyd-Warshall	Graph	Negative weights — rarely asked
A*	Graph	Game / pathfinding roles only
Optimized Fibonacci	Dynamic programming	Good for explaining memoization concept
Fractional Knapsack	Greedy	Understand the greedy vs DP contrast
Fast Exponentiation	Number theory	Useful for modular arithmetic problems
Sieve of Eratosthenes	Number theory	Prime-related problems only
Rabin-Karp	String matching	Rolling hash concept useful generally
Z-algorithm	String matching	Rarely asked outside competitive programming



AT ALL NETWORKS

We know the way!